

<https://www.sqldbachamps.com>

<https://github.com/PMSQLDBA/PraveenMadupu>

Telegram Group → SQLDBACloud-DevOps <https://t.me/+r2OYsT2qRZJkYWVl>

Praveen Madupu

Mb: +91 98661 30093

Database Administrator

AlwaysOn Availability Groups (AG) and the scenarios where the database log file fills up, why it happens, and detailed resolution steps along with real-time use cases.

Understanding the Scenario: AlwaysOn AG and Log File Full

In SQL Server, each database in an **AlwaysOn Availability Group (AG)** is typically set to **FULL recovery model** to support point-in-time recovery and transaction log replication to secondary replicas.

A **full transaction log** can happen when the log file grows uncontrollably and affects:

- Database writes → transactions stall
- Backups fail
- AG replication lag increases
- End-user applications experience delays

Key concept: In FULL recovery model, the **log file is not truncated** until a **log backup** occurs (or some internal truncation happens on secondary replicas).

Common Reasons for Log File Full in AlwaysOn AG

#	Cause	Explanation	Real-Time Symptom / DMV to Check
1	Log backup not running	Full recovery model requires regular transaction log backups. Without backups, logs never truncate.	DBCC SQLPERF(LOGSPACE) → log usage high, >90%
2	Long-running transactions	Open transactions prevent log truncation.	DBCC OPENTRAN → identifies active transactions
3	Replication lag / secondary replica issues	If AG secondary replica is disconnected, primary cannot truncate log.	sys.dm_hadr_database_replica_states → log_send_queue > 0
4	Heavy transaction workload	Bulk inserts, batch processing, index rebuilds can rapidly consume log space.	sys.dm_exec_requests → high log usage queries
5	Multiple large index rebuilds online	Online index operations use temp log space.	Check sys.dm_db_log_stats → high used_log_space_in_bytes
6	High VLF fragmentation	Very large log files with many Virtual Log Files (VLFs) cause inefficient space reuse.	DBCC LOGINFO → many small VLFs, slow truncation
7	Checkpoint / log truncation blocked	Active transactions or AG replication blocks checkpoint/log truncation.	sys.dm_os_waiting_tasks → waits for LOGMGR or WRITELOG
8	AG secondary in synchronous commit with high latency	Slow secondary can cause log send queue buildup → primary cannot truncate log	sys.dm_hadr_database_replica_states → high redo_queue_size

How to Diagnose Log File Full in AlwaysOn AG

Step 1: Check Log Space Usage

DBCC SQLPERF(LOGSPACE);

- Shows % log used per database.
- Identify the AG database with high log usage.

Step 2: Check Active Transactions

DBCC OPENTRAN ('YourDatabaseName');

- Shows if a long-running transaction is preventing log truncation.
- If yes → check app process, rollback if necessary.

Step 3: Check AG Log Send / Redo Queue

SELECT

**db_name(database_id) AS DBName,
log_send_queue_size,
redo_queue_size,
database_state_desc**

FROM sys.dm_hadr_database_replica_states;

- log_send_queue_size > 0 → primary is waiting for secondary.
- redo_queue_size > 0 → secondary is lagging.

Step 4: Check Log Fragmentation (VLFs)

DBCC LOGINFO ('YourDatabaseName');

- Too many VLFs (>1000) → inefficient truncation.
- Consider shrinking and regrowing log with proper size.

Resolution Steps

1 Immediate Actions (Log is full)

Action	Command / Notes
Backup transaction log	BACKUP LOG [YourDatabaseName] TO DISK = 'E:\Backup\YourDB.trn';
Free up log space	Use CHECKPOINT (careful if FULL recovery; best after log backup)
Kill runaway long transactions	KILL <SPID> (only if safe)
Add more log file space	ALTER DATABASE [YourDatabaseName] MODIFY FILE (NAME = YourDB_log, SIZE = 1GB);
Shrink log (temporary, not permanent solution)	DBCC SHRINKFILE('YourDB_log', 1024); (after backup/truncation)

2 Long-Term Fixes / Prevention

1. Ensure regular log backups

- Full recovery model requires log backups (e.g., every 15 mins or as per SLA)
- Automated SQL Agent jobs recommended.

2. **Monitor AG replication health**
 - Monitor log send queue and redo queue regularly
 - Alerts if secondary falls behind
3. **Break up large transactions**
 - Avoid massive single inserts or updates
 - Batch operations to reduce log consumption
4. **Maintain indexes wisely**
 - Rebuild indexes offline if possible
 - Online index rebuilds generate more log
5. **Proper log sizing**
 - Pre-size log files to expected workload
 - Avoid small auto-growth increments (<1 GB for large DB)
6. **VLF Management**
 - Resize log file to optimal size
 - Fewer large VLFs (e.g., 1–2 GB per VLF for large DBs)
 - Avoid too many small VLFs

Real-Time Use Case Scenarios

Scenario 1: Log Backup Missed

- **Symptom:** Log file 95% full, AG red queue growing
- **Cause:** SQL Agent log backup job failed last night
- **Action:** Run immediate log backup → log space truncated → monitor job success

Scenario 2: Secondary Replica Lag

- **Symptom:** Log_send_queue_size > 5 GB
- **Cause:** Secondary server under maintenance → lag
- **Action:** Resolve secondary connectivity, monitor redo queue, log truncation resumes

Scenario 3: Long-running ETL Batch

- **Symptom:** DB log grows 20 GB in 30 minutes → PAGEIOLATCH waits
- **Cause:** Huge batch inserts without commit
- **Action:** Break batch into smaller transactions → log stays under control

Summary Checklist (DBA Ready)

1. **Check log space:** DBCC SQLPERF(LOGSPACE)
2. **Check open transactions:** DBCC OPENTRAN
3. **Check AG queue:** sys.dm_hadr_database_replica_states
4. **Backup log immediately** → truncate
5. **Add log file size if needed**
6. **Review application transactions** → batch large operations
7. **Maintain log backups regularly**
8. **Monitor AG health** → alerts for lag
9. **Manage VLFs** → resize log if fragmented
10. **Avoid blocking long transactions** → rollback if safe

<https://www.sqldbachamps.com>

<https://github.com/PMSQLDBA/PraveenMadupu>

Telegram Group → SQLDBACloud-DevOps <https://t.me/+r2OYsT2qRZJkYWVl>

Praveen Madupu

Mb: +91 98661 30093

Database Administrator